

TCN 리뷰

Abstract

- 시퀀스 모델링의 기본 해법으로 RNN이 널리 쓰였지만 최근 합성곱 기반 아키텍처가 여러 작업에서 RNN을 능가한다는 결과가 보고됨
- 본 논문은 다양한 표준 시퀀스 과제에서 합성곱과 순환 아키텍처를 동일 조건으로 체계적으로 비교 평가
- 모든 과제에 동일하게 적용 가능한 단순한 Temporal Convolutional Network TCN을 제시
 - LSTM, GRU 같은 정석 RNN과 비교
- TCN은 폭넓은 작업과 데이터셋에서 RNN을 일관되게 능가하고 더 긴 유효 메모리를 보임
- 시퀀스 모델링에서 RNN이 기본이라는 통념을 재고할 필요가 있으며 합성곱 네트워크를 자연스러운 출발점으로 간주할 것을 제안

1. Introduction

- 교재와 강의 등에서 시퀀스 모델링은 전통적으로 RNN 중심으로 소개되어 왔음
- 최근 연구는 오디오 합성, 언어모델링, 기계번역 등에서 합성곱 기반 모델이 최신 성능을 달성
- 합성곱의 성공이 특정 도메인에 국한된 것인지 더 넓게 적용 가능한지에 대한 질문을 제기
- 다양한 RNN 벤치마크 과제에서 합성곱과 순환을 정면 비교
- 단순하고 일반적인 TCN을 정의해 모든 과제에 동일 구조로 적용
 - 결과적으로 TCN이 RNN보다 일관되게 우수하고 더 긴 기억을 유지

2. Background

- 합성곱 네트워크는 음성 인식과 NLP 등 시퀀스 처리에 오래전부터 적용되어 왔음
- RNN 계열은 시간에 따른 은닉 상태 전달로 시퀀스를 모델링
 - 학습 난이도 때문에 LSTM, GRU 등 변형이 널리 사용
- 많은 실증 연구가 RNN 변형들을 다양한 과제에서 비교 평가

- RNN과 CNN을 결합한 시도도 존재
- 시퀀스 생성 관점에서 합성곱과 순환을 폭넓게 동일 조건으로 비교한 연구는 부족
→ 본 논문이 이 공백을 메움

3. Temporal Convolutional Networks

- 목표는 단순성, 자동회귀, 예측, 긴 메모리를 동시에 달성하는 일반적 합성곱 구조를 제시하는 것
- TCN은 하나의 고정된 모델이 아니라 공통 특성을 공유하는 합성곱 계열을 가리키는 명칭
 - 인과적 합성곱을 사용해 미래 정보 누출을 방지
 - 입력 길이와 동일한 길이의 출력 시퀀스를 생성
 - RNN과 동일한 입출력 형태
 - 매우 깊은 네트워크와 팽창 합성곱 잔차 연결을 결합
 - 매우 긴 유효 이력을 확보

3.1 Sequence Modeling

- 각 시점의 출력을 과거 입력에만 의존하도록 하는 인과 제약을 만족하는 함수를 학습
- 자동회귀, 예측 등 많은 설정을 포괄
 - But, 전체 입력을 사용해 각 출력을 만드는 일반적 번역형 시퀀스 투 시퀀스 설정은 직접 다루지 않음

3.2 Causal Convolutions

- 1차원 완전 합성곱 네트워크를 사용
 - 모든 은닉층의 길이를 입력과 동일하게 유지하고 제로 패딩으로 길이를 맞춤
- 인과적 합성곱
 - 각 시점의 출력이 과거 시점의 은닉들에만 의존하도록 구성
 - 시간 지연 신경망에 제로 패딩을 더한 형태와 유사함

3.3 Dilated Convolutions

- 단순 인과 합성곱은 깊이에 비례해 볼 수 있는 이력이 제한
 - 팽창 합성곱으로 수용 영역을 크게 확장

- 층이 깊어질수록 간격을 기하급수적으로 늘려 적은 연산으로도 매우 긴 이력을 커버
- 필터 크기와 층 수 간격 스케줄을 조절해 과제에 필요한 메모리 길이에 맞춤

3.4 Residual Connections

- 잔차 블록 사용
 - 입력과 변환 결과를 더한 뒤, 활성화를 적용
- 한 블록은 팽창 인과 합성곱, 활성화, 드롭아웃을 두 번 반복하는 구조로 구성
- 가중치 정규화, 채널 단위 드롭아웃을 적용해 학습을 안정화
- 입력과 출력 차원이 다르면, 포인트와이즈 합성곱으로 모양을 맞춘 후 합산

3.5 Discussion

- 장점
 - 병렬성, 시점 간 순차 의존이 없으므로 긴 시퀀스를 한 번에 처리 가능
 - 수용 영역, 유연성, 층 수와 간격, 필터 크기 조절로 요구 메모리 길이에 쉽게 맞춤
 - 안정적 그래디언트, 시간 방향과 독립적인 역전파 경로로 폭주와 소실을 회피
 - 학습 메모리 효율, 게이트가 많은 RNN 대비 중간 결과 저장 요구가 낮음
 - 가변 길이 입력에 자연스럽게 대응 가능
- 단점
 - 평가 시, 과거 원시 시퀀스를 유효 이력 길이만큼 유지해야 하므로 메모리 요구가 커질 수 있음
 - 도메인 전이 시 필요한 이력 길이가 크게 달라지면, 간격과 필터 크기 등을 다시 조정해야 할 수 있음

4. Sequence Modeling Tasks

- Adding problem
 - 두 위치에 표시된 값 두 개를 합하도록 학습하는 합성 스트레스 테스트
- Sequential MNIST
 - 각 이미지를 길게 편 시퀀스로 제시하여 순차 분류
- Permuted MNIST
 - 순서를 임의로 섞어 난도를 높인 버전

- Copy memory
 - 시작 구간의 기호들을 구분 기호 이후 동일한 순서로 복사해야 하는 장기 기억 과제
- Polyphonic Music, JSB Chorales와 Nottingham
 - 다성음 악보 시퀀스의 다음 시점 음 스택 예측, 음향 모델링은 NLL로 평가
- PennTreebank PTB
 - 문자 수준과 단어 수준 언어모델링, 소규모 코퍼스
- WikiText 103
 - 대규모 위키 코퍼스, 희귀어 다수 포함
- LAMBADA
 - 소설 문맥 기반 마지막 단어 예측, 광범위한 문맥 활용 능력 평가
- text8
 - 문자 수준 언어모델링용 대규모 위키 데이터

5. Experiments

- 비교 대상
 - 제안한 일반 TCN과 LSTM, GRU, 단순 RNN을 동일 조건으로 비교함
- 공통 설정
 - 모든 과제에서 동일한 TCN 구조를 사용
 - 필요에 따라 깊이와 커널 크기만 조절해 필요한 수용 영역을 확보
 - 각 층의 간격은 깊어질수록 점차 크게 설정
 - 최적화는 Adam 기본으로 사용
 - 필요 시 그래디언트 클리핑 적용
 - RNN들은 유사한 파라미터 규모로 하이퍼파라미터 그리드 서치를 수행
 - 양쪽 모두 특수한 게이팅이나 스킵 연결 등, 추가 장치는 사용하지 않음
- 결과 개요
 - 표준 벤치마크 전반에서, 일반 TCN이 정석 RNN들을 일관되게 능가
 - 일부 과제에서 최신 특수 구조가 더 높을 수 있으나, 본 비교의 초점은 일반 구조 간 비교

- Synthetic Stress Tests
 - Adding problem에서 TCN은 빠르게 거의 완전한 해에 수렴
 - GRU는 느리지만 양호
 - LSTM과 단순 RNN은 열세를 보임
 - Sequential MNIST, Permuted MNIST에서 TCN이 수렴 속도와 최종 정확도에
서 우위를 보임
 - Copy memory에서 TCN은 길이가 길어져도 정답에 수렴
 - But, LSTM과 GRU는 길어지면 무작위 수준으로 붕괴
- Polyphonic Music and Language Modeling
 - JSB Chorales, Nottingham에서 별도 튜닝이 거의 없어도 TCN이 RNN을 유의
하게 앞섬
 - 단어 수준 언어모델링에서 작은 코퍼스에서는 최적화된 LSTM이 앞섬
 - 대규모 코퍼스와 넓은 문맥을 요구하는 과제에서는 TCN이 더 낮은 퍼플렉시티
를 보임
 - 문자 수준 언어모델링에서도 일반 TCN이 정규화된 LSTM과 GRU 같은 일반 구조
를 능가
- Memory Size of TCN and RNNs
 - Copy memory에서 길이를 늘려가며 측정
 - TCN은 긴 길이에서도 높은 정확도를 유지
 - LSTM과 GRU는 빠르게 붕괴
 - LAMBADA에서도 TCN이 더 낮은 퍼플렉시티를 보이며, 광범위한 문맥 활용력이
큼

6. Conclusion

- 단순한 TCN 아키텍처는 인과적 합성곱, 팽창 합성곱, 잔차 연결을 결합해 자동회귀 예
측과 긴 메모리를 구현
- 광범위한 시퀀스 과제에서 일반적인 RNN, LSTM, GRU보다 우수한 성능과 더 긴 실질
적 기억을 보임
- RNN의 이론적 무한 기억 이점은 실전에서 제한적이며 TCN이 같은 용량에서 더 긴 기
역을 보여줌

- 합성곱 네트워크를 시퀀스 모델링의 자연스러운 출발점이자 강력한 도구로 볼 것을 제안함